CENTRO UNIVERSITÁRIO CESUMAR MARINGÁ

Atividade de Estudo Programada (AEP)

ARTHUR VOLPATO MOREIRA

JOÃO VITOR DE MORAES GONGORA

PSE EM AÇÃO: SISTEMA DE PLANEJAMENTO E ACOMPANHAMENTO DE AÇÕES DE SAÚDE NA ESCOLA

Tema escolhido: Saúde Mental no ambiente escolar

MARINGÁ

2026

ARTHUR VOLPATO MOREIRA – 26004378-2
JOÃO VITOR DE MORAES GONGORA – 26003960-2

PSE EM AÇÃO: SISTEMA DE PLANEJAMENTO E ACOMPANHAMENTO DE AÇÕES DE SAÚDE NA ESCOLA

Atividade de Estudo Programada (AEP), 2ª entrega – Implementação e Apresentação, dos cursos de Engenharia de Software e Análise e Desenvolvimento de Sistemas, referente ao 2º semestre de 2026, apresentada à Unicesumar como requisito parcial para obtenção de nota no bimestre.

MARINGÁ
2026

1 INTRODUÇÃO

A primeira entrega desta Atividade de Estudo Programada (AEP) delimitou o problema, os objetivos, os requisitos e o planejamento do sistema PSE em Ação, proposto para apoiar a equipe intersetorial fictícia IntegraPSE no planejamento e no acompanhamento de ações coletivas do Programa Saúde na Escola (PSE). Naquela etapa, o foco esteve na compreensão do problema e na organização do planejamento, sem exigência de implementação completa.

Esta segunda entrega transforma aquele planejamento em uma aplicação funcional, desenvolvida em linguagem C e executada em terminal. O documento apresenta os requisitos atualizados após a implementação, o diagrama e a descrição dos casos de uso, as decisões de implementação adotadas, o registro dos testes realizados, o manual do usuário com capturas de tela e as considerações finais sobre resultados, limitações e possibilidades de melhoria.

O sistema mantém o recorte temático definido na primeira entrega — a saúde mental no ambiente escolar —, mas não restringe o cadastro a esse tema: a mesma estrutura permite registrar ações de alimentação saudável, cultura de paz, vacinação ou qualquer outra temática do programa, conforme exigido pelo desafio. Os limites éticos permanecem os mesmos: apenas dados fictícios e informações coletivas são utilizados, sem registro de nomes, diagnósticos, prontuários ou qualquer dado sensível individual dos estudantes.

2 REQUISITOS ATUALIZADOS

A implementação evidenciou ajustes pontuais no conjunto de requisitos definido na primeira entrega. Nenhum requisito foi removido: houve detalhamento de comportamentos já previstos, promoção de um requisito não funcional a requisito funcional e inclusão de restrições identificadas durante os testes. As tabelas 1 e 2 apresentam a versão consolidada, e a tabela 3 registra e justifica cada alteração.

2.1 Requisitos funcionais

Tabela 1 – Requisitos funcionais do sistema PSE em Ação.

Código	Requisito funcional
RF01	Permitir o cadastro de uma ação do PSE, informando código, escola, tema, data prevista, público-alvo, responsável, quantidade prevista de participantes e situação inicial.
RF02	Listar todas as ações cadastradas, apresentando os dados de cada registro de forma organizada e o total de registros armazenados.
RF03	Pesquisar ações por código, escola ou tema, aceitando trechos do texto informado e sem diferenciar letras maiúsculas de minúsculas.
RF04	Atualizar a situação de uma ação para planejada, realizada ou cancelada e, quando a ação for marcada como realizada, registrar a quantidade efetiva de participantes.
RF05	Gerar um resumo geral com a quantidade de ações planejadas, realizadas e canceladas, o total de participantes previstos e efetivos e o percentual de participação das ações realizadas em relação ao número previsto.
RF06	Validar as entradas do usuário, impedindo o armazenamento de códigos repetidos, campos obrigatórios vazios, datas inexistentes, quantidades negativas ou não numéricas e opções inexistentes no menu.

Fonte: elaboração própria (2026).

2.2 Requisitos não funcionais

Tabela 2 – Requisitos não funcionais do sistema PSE em Ação.

Código	Requisito não funcional
RNF01	Execução em terminal, sem interface gráfica, banco de dados ou bibliotecas externas, utilizando apenas a biblioteca padrão da linguagem C.
RNF02	Código organizado em funções com responsabilidades claras e nomes significativos, evitando concentrar toda a lógica na função main.
RNF03	Armazenamento em memória durante a execução, com limite de 100 ações cadastradas e tratamento explícito do limite de armazenamento.
RNF04	Mensagens claras em português, informando erros de entrada e confirmando cada operação realizada pelo usuário.

Código	Requisito não funcional
RNF05	Uso exclusivo de dados fictícios e coletivos, sem nomes, diagnósticos, prontuários ou qualquer dado clínico individual dos estudantes.

Fonte: elaboração própria (2026).

2.3 Alterações em relação à primeira entrega

A tabela 3 registra as mudanças aprovadas pelo grupo durante a implementação, com a justificativa de cada uma. As alterações foram mantidas dentro do escopo do desafio e não ampliaram o nível de complexidade previsto para o segundo semestre.

Tabela 3 – Atualização dos requisitos após a implementação.

Requisito	1ª entrega	2ª entrega	Justificativa
RF03	Pesquisar por código, escola ou tema.	Pesquisa por trecho, sem diferenciar maiúsculas de minúsculas, com total de registros encontrados.	Nos testes, a busca exata obrigava o usuário a digitar o nome completo da escola, o que dificultava a localização dos registros.
RF04	Atualizar situação e registrar participantes quando realizada.	Mantido, com aviso ao usuário quando a quantidade efetiva supera a prevista.	O aviso evita digitação incorreta sem impedir o registro de um público maior que o estimado, situação comum em ações coletivas.
RF05	Resumo com totais e percentual de participação.	Resumo com totais por situação, participantes previstos e efetivos, percentual das ações realizadas e percentual de ações concluídas.	O percentual calculado sobre todas as ações, incluindo as não realizadas, distorcia a leitura; o cálculo passou a considerar apenas as ações realizadas, que possuem quantidade efetiva registrada.
RF06 (novo)	Constava apenas como RNF01 (validação de entradas).	Promovido a requisito funcional.	A validação altera o comportamento observável do sistema e é percebida pelo usuário, caracterizando requisito funcional e não apenas atributo de qualidade.
RNF03 (novo)	Não previsto.	Limite de 100 ações em memória, com mensagem específica ao atingir o limite.	O desafio exige tratamento do limite de armazenamento; era necessário definir um valor explícito para que o tratamento pudesse ser implementado e testado.
RNF04 (novo)	Constava no escopo mínimo como “mensagens claras”.	Formalizado como requisito não funcional.	O item de escopo precisava ser verificável, o que exige um requisito associado.

Requisito	1ª entrega	2ª entrega	Justificativa
RNF01 e RNF02	Terminal (RNF02) e modularização (RNF03).	Renumerados como RNF01 e RNF02.	A renumeração decorre da promoção do antigo RNF01 (validação de entradas) a requisito funcional RF06.
RNF05 (novo)	O limite ético constava apenas na delimitação do escopo.	Formalizado como requisito não funcional de privacidade.	O limite ético precisava ser verificável para sustentar a coerência social da proposta.
RF01 (ajuste)	Fluxograma e pseudocódigo definiam a situação inicial como “planejada”.	O sistema solicita a situação inicial (1, 2 ou 3) no momento do cadastro.	Ações já realizadas são cadastradas depois de ocorrerem; fixar “planejada” obrigaria uma atualização imediata em parte dos registros.
RF06 (ajuste)	Validação prevista para campos vazios, código repetido e quantidade negativa.	Inclui também a validação da data no calendário.	Nos testes, datas inexistentes como 31/02 eram aceitas; a verificação do dia no mês resolveu o problema e foi incorporada ao requisito.

Fonte: elaboração própria (2026).

3 DIAGRAMA DE CASOS DE USO

O diagrama da figura 1 representa o ator principal e os casos de uso implementados. Os casos de uso UC01 a UC05 correspondem diretamente às funcionalidades disponíveis no menu principal, enquanto UC06 representa o comportamento de validação, incluído de forma obrigatória pelo cadastro (UC01) e pela atualização de situação (UC04).

A relação «include» foi utilizada porque a validação não é acionada diretamente pelo usuário: ela é executada sempre que os casos de uso de cadastro e de atualização leem os dados informados, caracterizando um comportamento comum reutilizado pelos dois casos de uso.

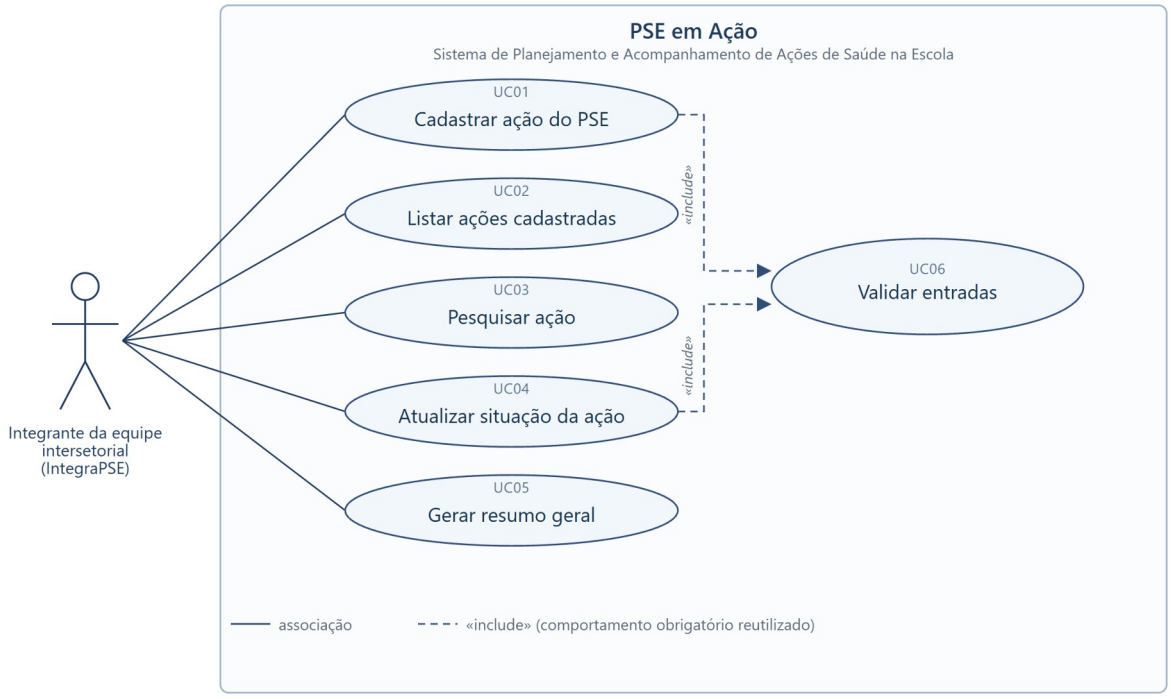


Figura 1 – Diagrama de casos de uso do sistema PSE em Ação. Fonte: elaboração própria (2026).

3.1 Atores e delimitação do sistema

O sistema possui um único ator humano: o integrante da equipe intersetorial (IntegraPSE), responsável por cadastrar, consultar, atualizar e consolidar as informações das ações. Não há outros atores, pois o sistema não se comunica com outros programas, não acessa banco de dados e não realiza integração com serviços externos.

Os limites do sistema abrangem o registro, a consulta, a atualização e a consolidação de informações sobre ações coletivas do PSE durante a execução do programa. Estão fora do escopo o armazenamento permanente dos dados, o controle de acesso por usuário, a emissão de relatórios em arquivo e qualquer processamento de informações individuais de estudantes.

4 DESCRIÇÃO DOS CASOS DE USO

As descrições a seguir apresentam, para cada caso de uso, o objetivo, o ator, as pré-condições, o fluxo principal, os fluxos alternativos e as pós-condições. Os fluxos alternativos correspondem exatamente às validações implementadas no código-fonte e exercitadas nos testes da seção 6.

4.1 UC01 – Cadastrar ação do PSE

Item	Descrição
Objetivo	Registrar uma nova ação coletiva do PSE no vetor de registros em memória, mantendo a integridade dos dados por meio da validação das entradas.
Ator	Integrante da equipe intersetorial (IntegraPSE).
Pré-condições	O sistema está em execução e o menu principal é exibido. Existe espaço disponível no vetor, ou seja, menos de 100 ações cadastradas.
Fluxo principal	1. O usuário seleciona a opção 1 do menu. 2. O sistema solicita o código da ação. 3. O sistema solicita escola, tema, data prevista, público-alvo e responsável. 4. O sistema solicita a quantidade prevista de participantes. 5. O sistema solicita a situação inicial (1 – planejada, 2 – realizada, 3 – cancelada). 6. Se a situação informada for realizada, o sistema solicita a quantidade efetiva de participantes. 7. O sistema valida cada entrada conforme UC06 e armazena o registro na próxima posição livre do vetor. 8. O sistema exibe mensagem de confirmação com o código cadastrado, a posição ocupada e a situação inicial.
Fluxos alternativos	A1. Código duplicado: o sistema informa que já existe ação com o código e retorna à leitura do código, sem cadastrar o registro. A2. Campo obrigatório vazio: o sistema informa que o campo é obrigatório e repete a leitura do mesmo campo. A3. Data inexistente ou fora do formato dd/mm/aaaa: o sistema informa a inconsistência e repete a leitura da data. A4. Quantidade não numérica, negativa ou superior a 9999: o sistema informa o intervalo aceito e repete a leitura. A5. Limite de armazenamento atingido: o sistema informa que as 100 posições estão ocupadas e encerra o caso de uso sem cadastrar. A6. Situação inicial fora das opções 1, 2 ou 3: o sistema informa o intervalo válido e repete a leitura.
Pós-condições	A ação é armazenada em memória e passa a aparecer na listagem, na pesquisa, na atualização de situação e no resumo geral.

Fonte: elaboração própria (2026).

4.2 UC02 – Listar ações cadastradas

Item	Descrição
Objetivo	Apresentar todas as ações armazenadas, com os dados completos de cada registro e o total de ações cadastradas.
Ator	Integrante da equipe intersetorial (IntegraPSE).
Pré-condições	O sistema está em execução e o menu principal é exibido.
Fluxo principal	1. O usuário seleciona a opção 2 do menu. 2. O sistema percorre o vetor de registros da posição 0 até a última posição ocupada. 3. Para cada registro, exibe número, código, escola, tema, data prevista, público-alvo, responsável, participantes previstos e realizados e situação. 4. Ao final, o sistema exibe o total de registros listados.
Fluxos alternativos	A1. Nenhuma ação cadastrada: o sistema informa que não há registros e orienta o usuário a utilizar a opção 1 para cadastrar a primeira ação.
Pós-condições	Nenhum dado é alterado; a operação é apenas de leitura.

Fonte: elaboração própria (2026).

4.3 UC03 – Pesquisar ação

Item	Descrição
Objetivo	Localizar rapidamente um ou mais registros a partir do código, de um trecho do nome da escola ou de um trecho do tema.
Ator	Integrante da equipe intersetorial (IntegraPSE).
Pré-condições	O sistema está em execução e o menu principal é exibido. Existe ao menos uma ação cadastrada.
Fluxo principal	1. O usuário seleciona a opção 3 do menu. 2. O sistema apresenta o submenu de pesquisa: 1 – por código, 2 – por escola, 3 – por tema, 0 – voltar ao menu. 3. O usuário escolhe o tipo de pesquisa e informa o termo desejado. 4. Para código, o sistema procura o registro correspondente; para escola ou tema, percorre o vetor comparando o termo com o campo escolhido, sem diferenciar maiúsculas de minúsculas. 5. O sistema exibe os registros encontrados e a quantidade de resultados.

Item	Descrição
Fluxos alternativos	A1. Nenhum registro encontrado: o sistema informa que não há ações para a busca informada. A2. Código inexistente: o sistema informa que não existe registro com o código informado. A3. Submenu com opção 0: a pesquisa é cancelada e o sistema retorna ao menu principal. A4. Nenhuma ação cadastrada: o sistema informa a ausência de registros e encerra o caso de uso.
Pós-condições	Nenhum dado é alterado; a operação é apenas de consulta.

Fonte: elaboração própria (2026).

4.4 UC04 – Atualizar situação da ação

Item	Descrição
Objetivo	Alterar a situação de uma ação já cadastrada e registrar a quantidade efetiva de participantes quando a ação for marcada como realizada.
Ator	Integrante da equipe intersetorial (IntegraPSE).
Pré-condições	O sistema está em execução e o menu principal é exibido. Existe ao menos uma ação cadastrada.
Fluxo principal	1. O usuário seleciona a opção 4 do menu. 2. O sistema solicita o código da ação. 3. O sistema localiza o registro e exibe seus dados atuais. 4. O sistema apresenta as opções de nova situação: 1 – planejada, 2 – realizada, 3 – cancelada, 0 – cancelar a atualização. 5. Se a nova situação for realizada, o sistema solicita a quantidade efetiva de participantes. 6. O sistema atualiza o registro e exibe mensagem de confirmação com a nova situação.
Fluxos alternativos	A1. Registro inexistente: o sistema informa que não existe ação com o código informado e orienta a conferir o código na listagem. A2. Opção 0: a atualização é cancelada sem alteração no registro. A3. Quantidade efetiva maior que a prevista: o sistema registra o valor e exibe aviso de que a quantidade supera a prevista. A4. Nova situação planejada ou cancelada: o sistema zera a quantidade efetiva, pois não há participantes confirmados fora da situação realizada. A5. Opção de situação fora do intervalo 0 a 3: o sistema informa o intervalo válido e repete a leitura.
Pós-condições	A situação e, quando aplicável, a quantidade efetiva do registro ficam atualizadas e passam a ser consideradas no resumo geral.

Fonte: elaboração própria (2026).

4.5 UC05 – Gerar resumo geral

Item	Descrição
Objetivo	Consolidar os dados das ações cadastradas, apresentando a quantidade por situação, os totais de participantes e o percentual de participação em relação ao previsto.
Ator	Integrante da equipe intersetorial (IntegraPSE).
Pré-condições	O sistema está em execução e o menu principal é exibido. Existe ao menos uma ação cadastrada.
Fluxo principal	1. O usuário seleciona a opção 5 do menu. 2. O sistema percorre o vetor e conta as ações planejadas, realizadas e canceladas. 3. O sistema soma as quantidades previstas de todos os registros e as quantidades previstas e efetivas das ações realizadas. 4. O sistema calcula o percentual de participação das ações realizadas (efetivo dividido pelo previsto) e o percentual de ações concluídas em relação ao total. 5. O sistema exibe o resumo com os valores calculados.
Fluxos alternativos	A1. Nenhuma ação cadastrada: o sistema informa que não há dados para gerar o resumo. A2. Nenhuma ação realizada: o sistema informa que não há ações realizadas para calcular o percentual de participação.
Pós-condições	Nenhum dado é alterado; a operação é apenas de consolidação.

Fonte: elaboração própria (2026).

4.6 UC06 – Validar entradas (incluído por UC01 e UC04)

Item	Descrição
Objetivo	Impedir que dados inválidos sejam armazenados, garantindo a consistência dos registros e informando o usuário sobre cada inconsistência.
Ator	Sistema (caso de uso incluído, executado durante UC01 e UC04).
Pré-condições	Um dos casos de uso que incluem UC06 está em execução. O sistema acabou de ler um valor informado pelo usuário.

Item	Descrição
Fluxo principal	1. O sistema verifica se o campo obrigatório está vazio ou contém apenas espaços. 2. O sistema verifica o formato e o intervalo do valor, conforme o tipo do campo: texto, data, código ou número inteiro. 3. Para o código, verifica também se já existe registro com o mesmo valor. 4. Se o valor for válido, o sistema o devolve ao caso de uso que o solicitou. 5. Se o valor for inválido, o sistema exibe mensagem específica e repete a leitura do mesmo campo.
Fluxos alternativos	A1. Texto acima do tamanho do campo: o sistema informa o limite de caracteres e repete a leitura. A2. Código com caracteres diferentes de letras, números, hífen ou sublinhado: o sistema informa o formato aceito e repete a leitura. A3. Data com dia inexistente para o mês, incluindo fevereiro de ano bissexto: o sistema informa que a data é inválida e repete a leitura.
Pós-condições	O caso de uso que inclui UC06 recebe um valor válido e prossegue com a operação; nenhum dado inválido é armazenado.

Fonte: elaboração própria (2026).

5 IMPLEMENTAÇÃO DO SISTEMA

O sistema foi implementado em um único arquivo-fonte, pse_em_acao.c, composto pelas declarações de tipos, pelas funções utilitárias de entrada, pelas funções de apoio ao domínio do problema e pelas cinco funcionalidades do menu. A função main limita-se a controlar o laço do menu e a chamar a funcionalidade escolhida, conforme o requisito não funcional RNF02.

5.1 Estrutura de dados

Os registros são armazenados em um vetor de estruturas. A struct Acao agrupa os dados de uma ação e a struct Sistema reúne o vetor e o contador de posições ocupadas, evitando o uso de variáveis globais: o estado do sistema é passado às funções por referência. A tabela 4 descreve os campos da struct Acao.

Tabela 4 – Campos da estrutura Acao.

Campo	Tipo	Descrição
codigo	char[10]	Código de identificação da ação, único no sistema (ex.: PSE-001).

Campo	Tipo	Descrição
escola	char[60]	Nome da escola em que a ação será realizada.
tema	char[50]	Temática do PSE trabalhada na ação.
dataPrevista	char[11]	Data prevista no formato dd/mm/aaaa, validada no calendário.
publicoAlvo	char[50]	Público-alvo da ação (turma ou segmento atendido).
responsavel	char[50]	Profissional ou equipe responsável pela ação.
quantidadePrevista	int	Quantidade prevista de participantes.
quantidadeRealizada	int	Quantidade efetiva de participantes, registrada quando a ação é marcada como realizada.
situacao	int	Situação da ação: 1 – planejada, 2 – realizada, 3 – cancelada.

Fonte: elaboração própria (2026).

A definição das estruturas e do vetor de registros é apresentada no quadro 1.

Quadro 1 – Definição das estruturas Acao e Sistema.

```
/* Registro de uma ação coletiva do PSE */
struct Acao {
    char codigo[TAM_CODIGO];
    char escola[TAM_ESCOLA];
    char tema[TAM_TEMA];
    char dataPrevista[TAM_DATA];
    char publicoAlvo[TAM_PUBLICO];
    char responsavel[TAM_RESPONSAVEL];
    int quantidadePrevista;
    int quantidadeRealizada;
    int situacao;
};

/* Vetor de registros em memória e controle de ocupação */
struct Sistema {
    struct Acao acoes[MAX_ACOES];
    int total;
};
```

Fonte: elaboração própria (2026).

Os campos de texto utilizam vetores de tamanho fixo, definidos por constantes, o que dispensa alocação dinâmica de memória e mantém o código compatível com o conteúdo trabalhado no semestre. A constante MAX_ACOES define o limite de armazenamento exigido pelo requisito não funcional RNF03.

5.2 Organização das funções

A tabela 5 apresenta as funções que compõem o programa e a responsabilidade de cada uma. As funções de entrada foram agrupadas em uma única camada, de modo que as funcionalidades do menu não precisem conhecer detalhes de leitura do teclado.

Tabela 5 – Organização das funções do programa.

Função	Responsabilidade
main	Exibir o menu em laço, ler a opção e chamar a funcionalidade correspondente.
exibirMenu	Apresentar o cabeçalho e as opções do menu principal.
cadastrarAcao	Executar o cadastro de uma ação, com todas as validações de entrada.
listarAcoes	Percorrer o vetor e exibir todos os registros cadastrados.
pesquisarAcao	Executar a pesquisa por código, escola ou tema.
atualizarSituacao	Localizar um registro, alterar a situação e registrar participantes.
gerarResumo	Consolidar totais por situação e calcular os percentuais.
buscarIndicePorCodigo	Retornar a posição do registro com o código informado ou -1.
exibirAcao	Imprimir os dados de um registro em formato de bloco.
nomeSituacao	Converter o código numérico da situação em texto para exibição.
lerTexto, lerInteiro, lerData, lerCodigo	Ler e validar cada tipo de campo, repetindo a leitura enquanto o valor for inválido.
lerLinha	Ler uma linha inteira do teclado e descartar o excedente.
codigoValido, dataValida, textoVazio	Validar formato de código, existência da data e preenchimento de campos obrigatórios.
contemTexto, compararSemDiferenciar, converterMinusculo	Comparar textos sem diferenciar maiúsculas de minúsculas.
formatarPercentual	Formatar o percentual com uma casa decimal e vírgula.
pausar	Aguardar a confirmação do usuário antes de retornar ao menu.
configurarConsole	Ajustar a saída do console para UTF-8 no Windows.

Fonte: elaboração própria (2026).

5.3 Decisões de implementação

Algumas decisões foram tomadas durante a codificação para atender aos requisitos e evitar problemas comuns no uso de terminal. As principais estão descritas a seguir.

Leitura por linha em vez de scanf: todas as entradas passam pela função lerLinha, que consome a linha inteira do teclado e descarta o excedente. Com scanf, o caractere de nova linha permanece no buffer e contamina as leituras seguintes, problema que aparecia nos primeiros testes com campos de texto contendo espaços.

Validação com repetição: as funções lerTexto, lerInteiro, lerData e lerCodigo repetem a leitura até que o valor seja válido, exibindo mensagem específica a cada tentativa. O programa não é encerrado por erro de digitação, o que torna o uso mais seguro para o público-alvo.

Comparação de textos sem diferenciar maiúsculas: as funções converterMinusculo e contemTexto permitem que a pesquisa por escola ou tema encontre registros mesmo quando o usuário digita o termo com capitalização diferente da utilizada no cadastro.

Ausência de variáveis globais: o vetor de registros e o contador de ocupação estão dentro da struct Sistema, passada por ponteiro para as funções de escrita e por ponteiro constante para as funções de leitura, o que deixa explícito quais funções alteram o estado do programa.

Validação de datas no calendário: dataValida verifica o formato, o intervalo de ano, o mês e a quantidade de dias de cada mês, incluindo o dia 29 de fevereiro em anos bissextos. Assim, datas como 31/02 são recusadas antes do armazenamento.

Compilação em C99 e saída em UTF-8: o código foi escrito para o padrão C99 e utiliza SetConsoleOutputCP(65001) no Windows para que os acentos sejam exibidos corretamente no terminal, sem alterar o funcionamento em outros sistemas operacionais.

5.4 Trechos de código

Os quadros a seguir apresentam trechos que ilustram as decisões descritas. O código completo está disponível no repositório do projeto.

A leitura de números inteiros combina strtol com a verificação do conteúdo restante da linha, o que recusa entradas como “12abc” e valores fora do intervalo permitido.

Quadro 2 – Leitura e validação de números inteiros.

```
static int lerInteiro(const char *rotulo, int minimo, int maximo) {
    char linha[TAM_LINHA];
    char *fim;
    long valor;

    for (;;) {
        printf("%s", rotulo);
        fflush(stdout);
        lerLinha(linha, sizeof(linha));

        if (feof(stdin)) {
            return minimo;
        }

        valor = strtol(linha, &fim, 10);

        while (*fim == ' ' || *fim == '\t') {
            fim++;
        }

        if (fim == linha || *fim != '\0') {
            printf("  [!] Entrada inválida: digite apenas números.\n\n");
            continue;
        }

        if (valor < minimo || valor > maximo) {
            printf("  [!] Valor fora do intervalo permitido (%d a %d).\n\n",
                   minimo, maximo);
            continue;
        }

        return (int)valor;
    }
}
```

Fonte: elaboração própria (2026).

A validação de datas verifica o formato e a existência do dia no calendário, incluindo fevereiro de ano bissexto.

Quadro 3 – Validação de datas no formato dd/mm/aaaa.

```
static int dataValida(const char *data) {
    int diasPorMes[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    int dia, mes, ano, i;

    if (strlen(data) != 10) {
        return 0;
    }

    for (i = 0; i < 10; i++) {
        if (i == 2 || i == 5) {
            if (data[i] != '/') {
                return 0;
            }
        } else if (!isdigit((unsigned char)data[i])) {
            return 0;
        }
    }

    dia = (data[0] - '0') * 10 + (data[1] - '0');
    mes = (data[3] - '0') * 10 + (data[4] - '0');
    ano = (data[6] - '0') * 1000 + (data[7] - '0') * 100
          + (data[8] - '0') * 10 + (data[9] - '0');

    if (mes < 1 || mes > 12) {
        return 0;
    }

    if (ano < 2000 || ano > 2100) {
        return 0;
    }

    if ((ano % 4 == 0 && ano % 100 != 0) || ano % 400 == 0) {
        diasPorMes[1] = 29;
    }

    if (dia < 1 || dia > diasPorMes[mes - 1]) {
        return 0;
    }

    return 1;
}
```

Fonte: elaboração própria (2026).

A busca por código é centralizada em uma única função, utilizada pelo cadastro (para impedir duplicidade), pela pesquisa e pela atualização de situação, evitando a repetição da mesma lógica em três pontos do programa.

Quadro 4 – Busca de registro pelo código.

```
static int buscarIndicePorCodigo(const struct Sistema *sis, const char *codigo) {
    int i;

    for (i = 0; i < sis->total; i++) {
        if (compararSemDiferenciar(sis->acoes[i].codigo, codigo) == 0) {
            return i;
        }
    }

    return -1;
}
```

Fonte: elaboração própria (2026).

No resumo geral, o percentual de participação é calculado apenas sobre as ações realizadas, que possuem quantidade efetiva registrada, e formatado com vírgula decimal.

Quadro 5 – Cálculo do percentual de participação.

```
    if (previstoRealizadas > 0) {
        percentual = (100.0 * efetivoRealizadas) / previstoRealizadas;
    }
```

Fonte: elaboração própria (2026).

6 TESTES REALIZADOS

Os testes foram executados com o objetivo de verificar o funcionamento das seis funcionalidades mínimas, o comportamento das validações e o tratamento das situações de erro previstas no desafio: registro inexistente, código duplicado e limite de armazenamento atingido.

6.1 Ambiente e compilação

Os testes foram realizados em microcomputador com sistema operacional Windows 11, utilizando o compilador gcc 6.3.0 (MinGW) e o terminal do sistema. A compilação foi executada com as opções -std=c99 -Wall -Wextra, que habilitam os principais avisos do compilador, e não produziu erros nem advertências, conforme o comando a seguir.

```
gcc -std=c99 -Wall -Wextra -o pse_em_acao pse_em_acao.c
```

Fonte: elaboração própria (2026).

O cenário de limite de armazenamento foi verificado com o cadastro automatizado de 100 ações fictícias, seguido da tentativa de cadastrar o 101º registro, que deve ser recusado com mensagem específica.

6.2 Casos de teste

A tabela 6 apresenta os casos de teste executados, com a entrada utilizada, o resultado esperado e a situação observada. Todos os casos foram executados sobre a versão final do código-fonte.

Tabela 6 – Casos de teste executados.

ID	Caso de teste	Entrada	Resultado esperado	Situação
CT01	Cadastro com dados válidos	PSE-001, escola, tema, data 15/09/2026, público, responsável, 40 participantes, situação 1	Registro armazenado e mensagem de confirmação	Conforme
CT02	Cadastro com código duplicado	Código PSE-001 já cadastrado	Mensagem de código duplicado e nova leitura do código	Conforme
CT03	Campo obrigatório vazio	Escola preenchida apenas com espaços	Mensagem de campo obrigatório e nova leitura	Conforme
CT04	Data inexistente	31/02/2026	Mensagem de data inválida e nova leitura	Conforme
CT05	Data em formato inválido	2026-09-15	Mensagem de data inválida e nova leitura	Conforme
CT06	Quantidade não numérica	abc	Mensagem de entrada inválida e nova leitura	Conforme
CT07	Quantidade negativa	-8	Mensagem de valor fora do intervalo (0 a 9999)	Conforme
CT08	Opção inexistente no menu	7	Mensagem de opção inexistente e retorno ao menu	Conforme
CT09	Atualização de registro inexistente	PSE-999	Mensagem de ação não encontrada	Conforme
CT10	Atualização para realizada	PSE-002, situação 2, 38 participantes	Situação alterada e quantidade efetiva registrada	Conforme
CT11	Resumo geral com cinco ações	Duas planejadas, duas realizadas e uma cancelada	Totais por situação, 205 previstos, 84 efetivos e 93,3% de participação	Conforme
CT12	Pesquisa por tema	Termo “mental”	Três registros encontrados	Conforme
CT13	Pesquisa sem resultados	Termo inexistente	Mensagem de nenhuma ação encontrada	Conforme

ID	Caso de teste	Entrada	Resultado esperado	Situação
CT14	Limite de armazenamento	100 registros cadastrados e tentativa do 101º	Mensagem de limite atingido, sem cadastro	Conforme
CT15	Encerramento do sistema	Opção 0	Mensagem de encerramento e fim da execução	Conforme

Fonte: elaboração própria (2026).

6.3 Resultados dos testes

Os quinze casos de teste apresentaram o resultado esperado. As validações impediram o armazenamento de registros com código repetido, campos vazios, datas inexistentes e quantidades inválidas; o cenário de limite recusou o 101º registro com a mensagem prevista; e o resumo geral apresentou valores consistentes com os dados cadastrados durante a sessão de testes.

Também foram observadas duas situações de melhoria durante os testes, já incorporadas ao código: o aviso quando a quantidade efetiva supera a prevista e o cancelamento explícito da atualização de situação, que evita alterações acidentais quando o código é digitado por engano.

6.4 Relação entre o planejamento e a implementação

O cronograma de sprints definido na primeira entrega previa, para as semanas 5 a 8, a conclusão do código, dos casos de uso, do manual do usuário, do documento final e do vídeo. A tabela 7 compara o que foi planejado com o que foi efetivamente entregue nesta etapa, evidenciando o vínculo entre o planejamento e a implementação.

Tabela 7 – Planejamento das sprints e resultado da 2ª entrega.

Semana	Previsto na 1ª entrega	Realizado na 2ª entrega
5	Código completo no repositório; diagrama de casos de uso; manual do usuário.	Código em codigo/pse_em_acao.c, compilando sem avisos; diagrama em diagramas/casos_de_uso.png; manual na seção 7.
6	Descrição dos casos de uso; atualização dos requisitos; capturas de tela.	Descrição de UC01 a UC06 na seção 4; requisitos atualizados na seção 2, com a tabela 3; capturas de tela nas figuras 2 a 11.
7	Considerações finais; apresentação do problema; documento final em Word ou PDF.	Considerações finais na seção 8; roteiro do vídeo elaborado; documento final gerado em Word e em PDF.

Semana	Previsto na 1ª entrega	Realizado na 2ª entrega
8	Apresentação em vídeo com duração máxima de 7 minutos.	Gravação e edição em andamento pelos dois integrantes; o link será inserido na seção 9 assim que o vídeo for publicado.

Fonte: elaboração própria (2026).

As semanas 9 a 15 do cronograma original correspondem à edição do vídeo, à conferência dos critérios avaliativos das duas etapas e à revisão final do conteúdo. Essas atividades seguem em execução e não alteram os artefatos já concluídos.

7 MANUAL DO USUÁRIO

7.1 Objetivo do manual

Este manual orienta a compilação, a execução e o uso do sistema PSE em Ação por um integrante da equipe intersetorial. O sistema é executado em terminal e não requer instalação de programas adicionais além do compilador.

7.2 Requisitos para execução

Para compilar e executar o sistema são necessários: um microcomputador com Windows, Linux ou macOS; o compilador gcc instalado (no Windows, por meio do MinGW); e um terminal ou prompt de comando. O sistema utiliza apenas a biblioteca padrão da linguagem C, não dependendo de bibliotecas externas, banco de dados ou conexão com a internet.

7.3 Como compilar

O código-fonte está no arquivo pse_em_acao.c, disponível na pasta codigo do repositório do projeto. No terminal, deve-se acessar a pasta do arquivo e executar o comando de compilação correspondente ao sistema operacional.

Windows (MinGW):

```
gcc -std=c99 -Wall -Wextra -o pse_em_acao.exe pse_em_acao.c
```

Linux ou macOS:

```
gcc -std=c99 -Wall -Wextra -o pse_em_acao pse_em_acao.c
```

Fonte: elaboração própria (2026).

A compilação gera o arquivo executável pse_em_acao.exe (Windows) ou pse_em_acao (Linux/macOS). Caso o compilador não esteja instalado, deve-se instalá-lo previamente; no Windows, o MinGW inclui o gcc e o terminal necessário.

7.4 Como executar

Após a compilação, o programa é iniciado pelo terminal, na mesma pasta do executável.

Windows:

```
pse_em_acao.exe
```

Linux ou macOS:

```
./pse_em_acao
```

Fonte: elaboração própria (2026).

Ao iniciar, o sistema exibe uma mensagem de boas-vindas e o menu principal. A navegação é feita pela digitação do número da opção desejada, seguida da tecla ENTER. Ao final de cada operação, o sistema aguarda a tecla ENTER para retornar ao menu, o que permite ler as mensagens com calma.

7.5 Opções do menu

Tabela 8 – Opções do menu principal.

Opção	Funcionalidade	O que faz
1	Cadastrar ação	Solicita os dados da ação e armazena o registro em memória.
2	Listar ações cadastradas	Exibe todos os registros com seus dados e o total de ações.
3	Pesquisar ação	Permite localizar registros por código, escola ou tema.
4	Atualizar situação da ação	Altera a situação e registra a quantidade efetiva de participantes.
5	Gerar resumo geral	Consolida totais por situação, participantes e percentuais.
0	Encerrar o sistema	Finaliza a execução do programa.

Fonte: elaboração própria (2026).

Além das opções do menu principal, o sistema utiliza o número 0 em dois contextos: no submenu de pesquisa, cancela a pesquisa e retorna ao menu; na atualização de situação, cancela a alteração sem modificar o registro.

7.6 Exemplos de uso

7.6.1 Tela inicial e menu principal

Ao executar o programa, o usuário visualiza a mensagem de boas-vindas e o menu com as seis opções. A figura 2 apresenta essa tela.

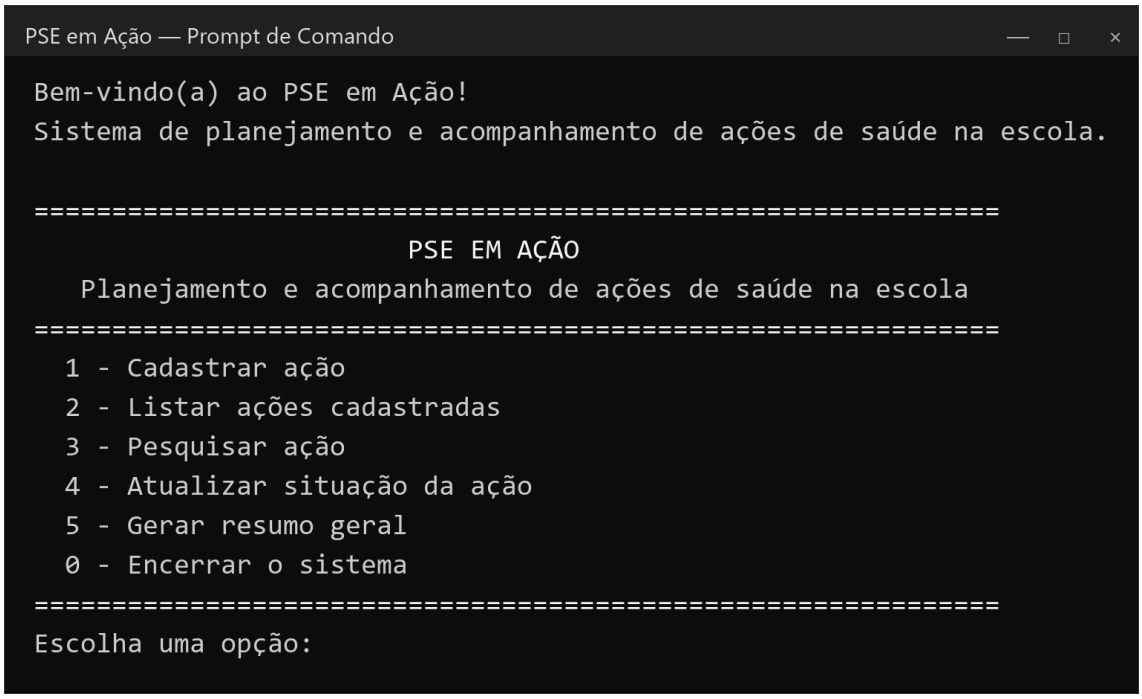


Figura 2 – Tela inicial e menu principal do sistema. Fonte: elaboração própria (2026).

7.6.2 Cadastro de uma ação

Para cadastrar uma ação, o usuário digita 1 e informa os dados solicitados. O sistema confirma o cadastro apresentando o código, a posição ocupada no vetor e a situação inicial. A figura 3 mostra o cadastro da ação PSE-001.

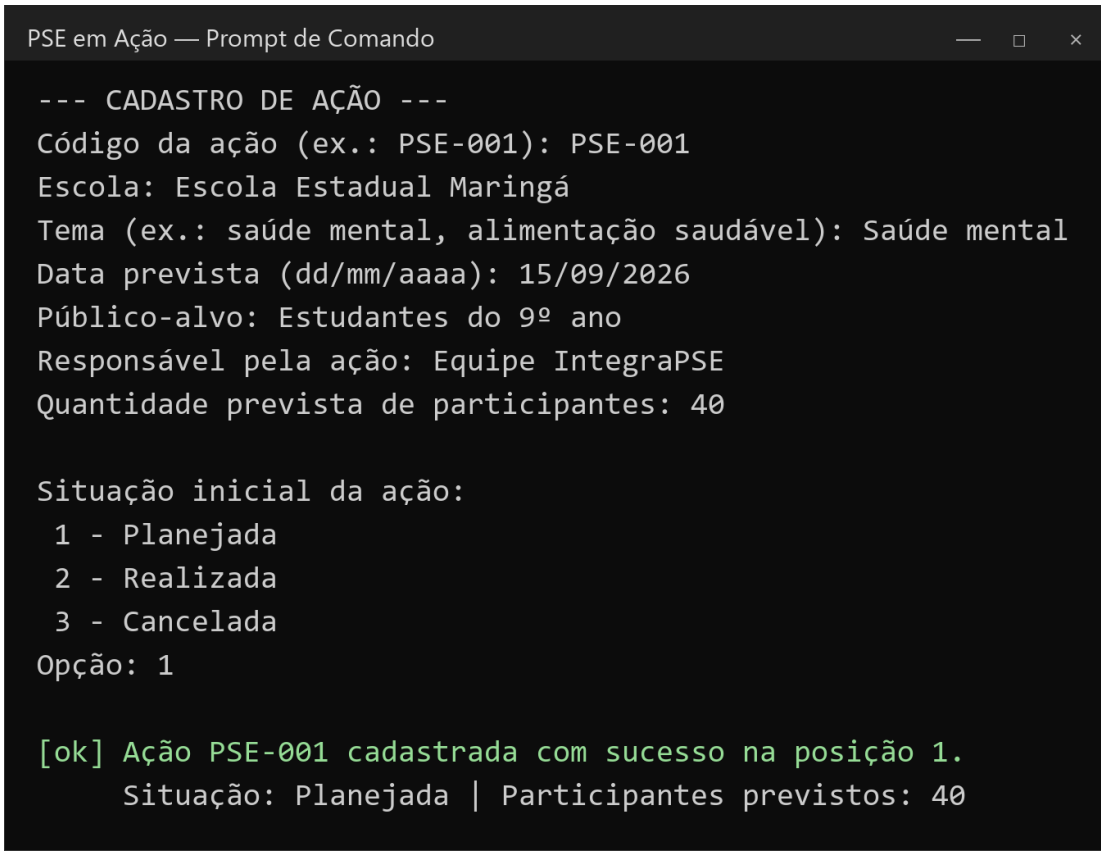


Figura 3 – Cadastro de uma ação do PSE. Fonte: elaboração própria (2026).

7.6.3 Validação das entradas

Quando um dado inválido é informado, o sistema apresenta uma mensagem específica e repete a leitura do mesmo campo, sem perder os dados já digitados. A figura 4 apresenta a recusa de um código duplicado, de um campo obrigatório vazio e de uma data inexistente.

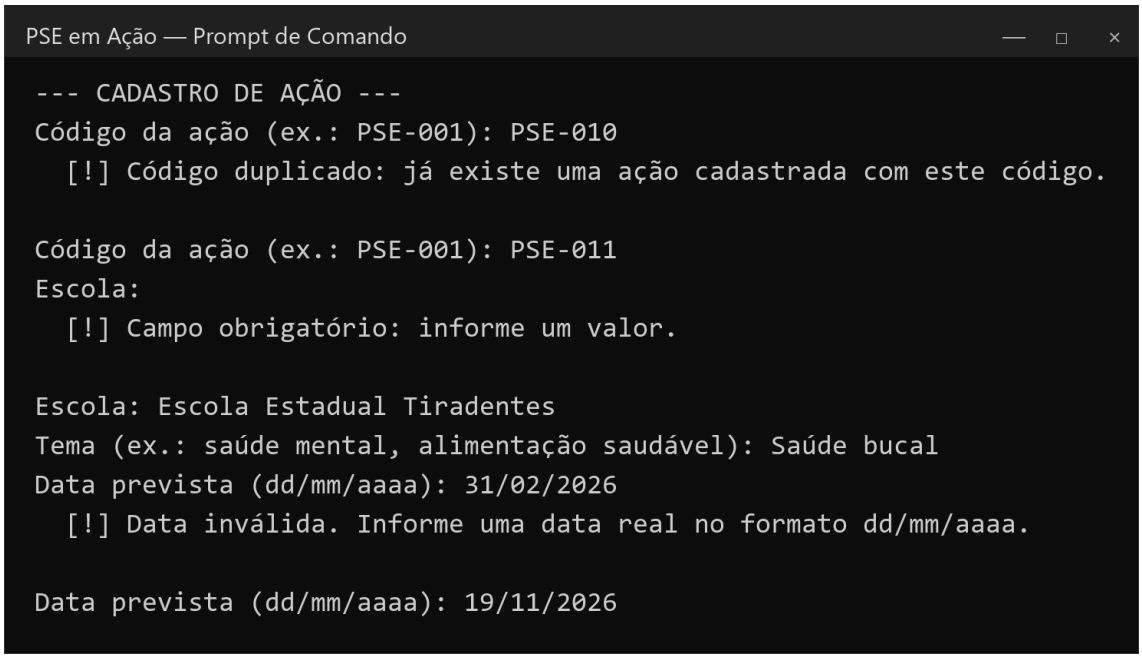


Figura 4 – Validação de código duplicado, campo obrigatório e data inválida. Fonte: elaboração própria (2026).

A figura 5 apresenta a validação da quantidade prevista, que recusa valores não numéricos e negativos, e a validação da situação inicial, que aceita apenas as opções 1, 2 ou 3.

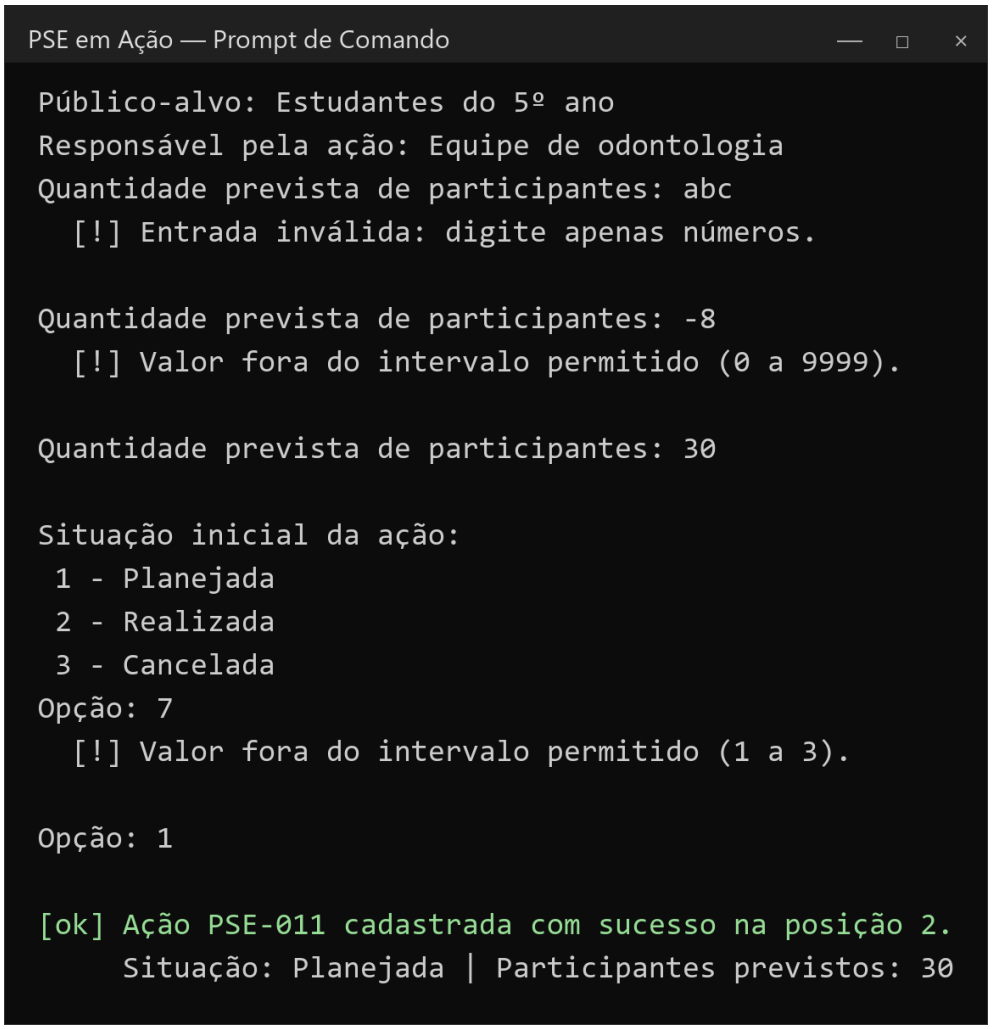


Figura 5 – Validação de quantidade e de opção do menu. Fonte: elaboração própria (2026).

7.6.4 Listagem das ações

A opção 2 apresenta todas as ações cadastradas, uma após a outra, com a situação e as quantidades de participantes de cada registro. As figuras 6 e 7 mostram a listagem de cinco ações cadastradas durante a sessão de demonstração.

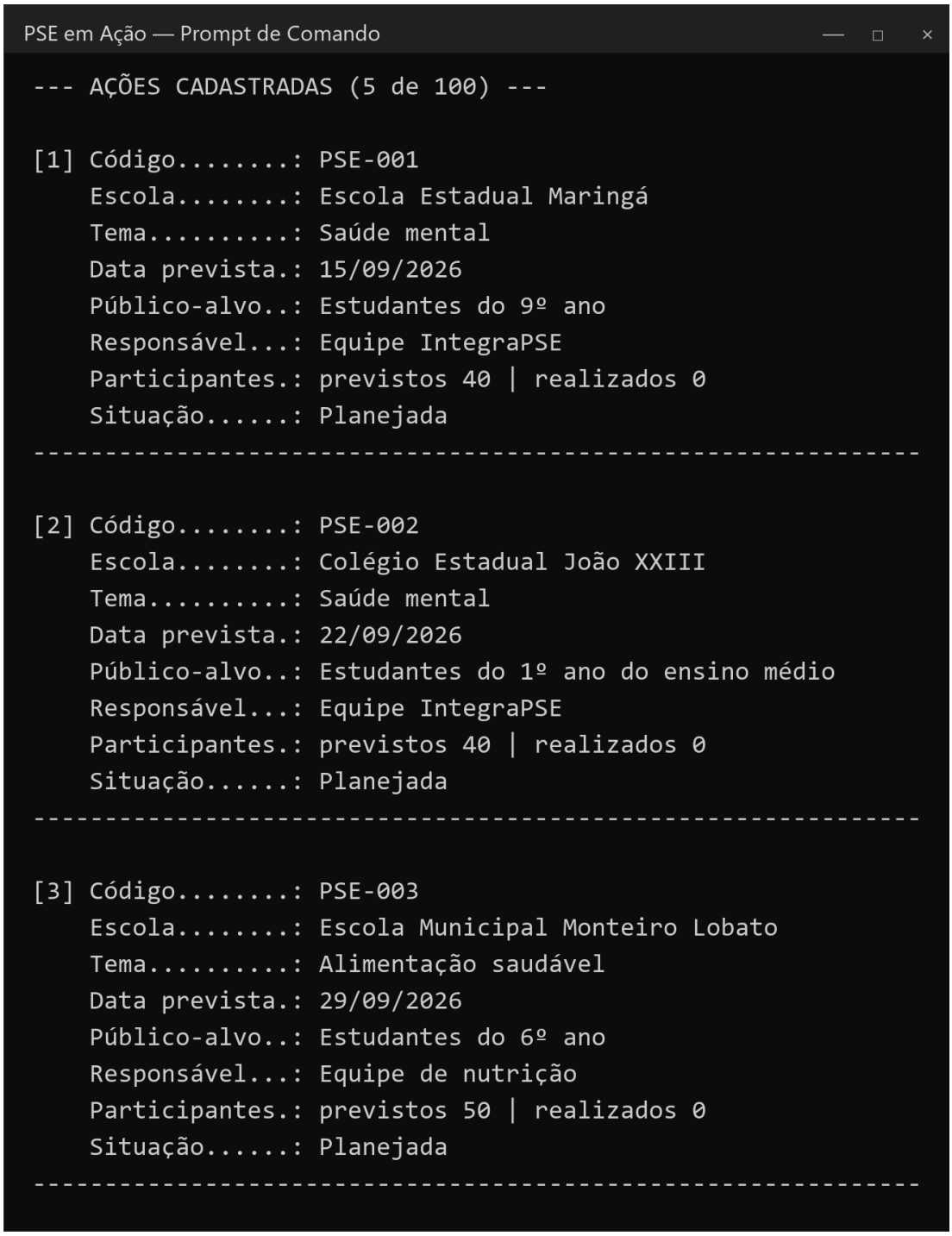


Figura 6 – Listagem das ações cadastradas, registros 1 a 3. Fonte: elaboração própria (2026).

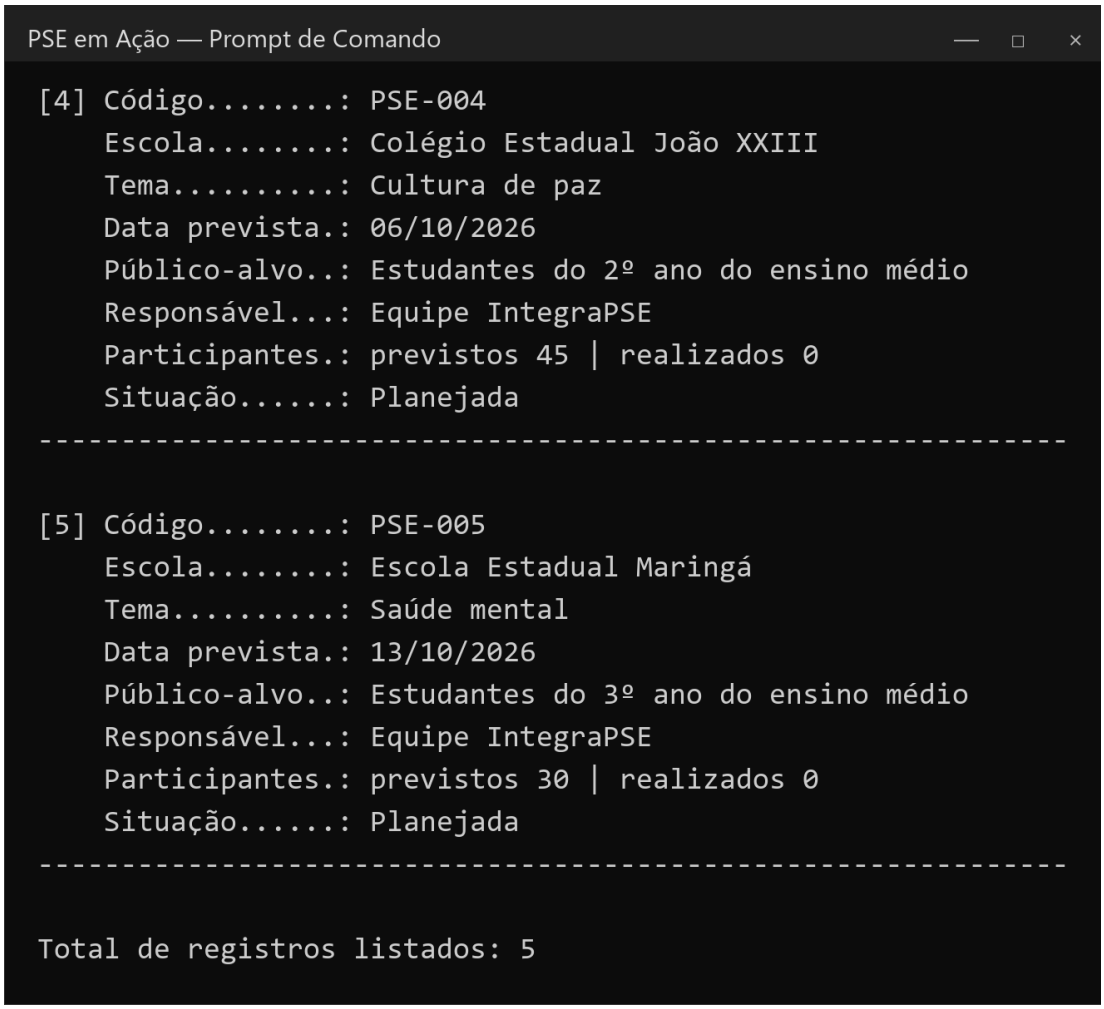


Figura 7 – Listagem das ações cadastradas, registros 4 e 5. Fonte: elaboração própria (2026).

7.6.5 Pesquisa por tema

Na opção 3, o usuário escolhe o critério de pesquisa e informa o termo desejado. A busca aceita trechos do texto e não diferencia letras maiúsculas de minúsculas. A figura 8 mostra a pesquisa pelo termo “mental”, que retorna três registros.

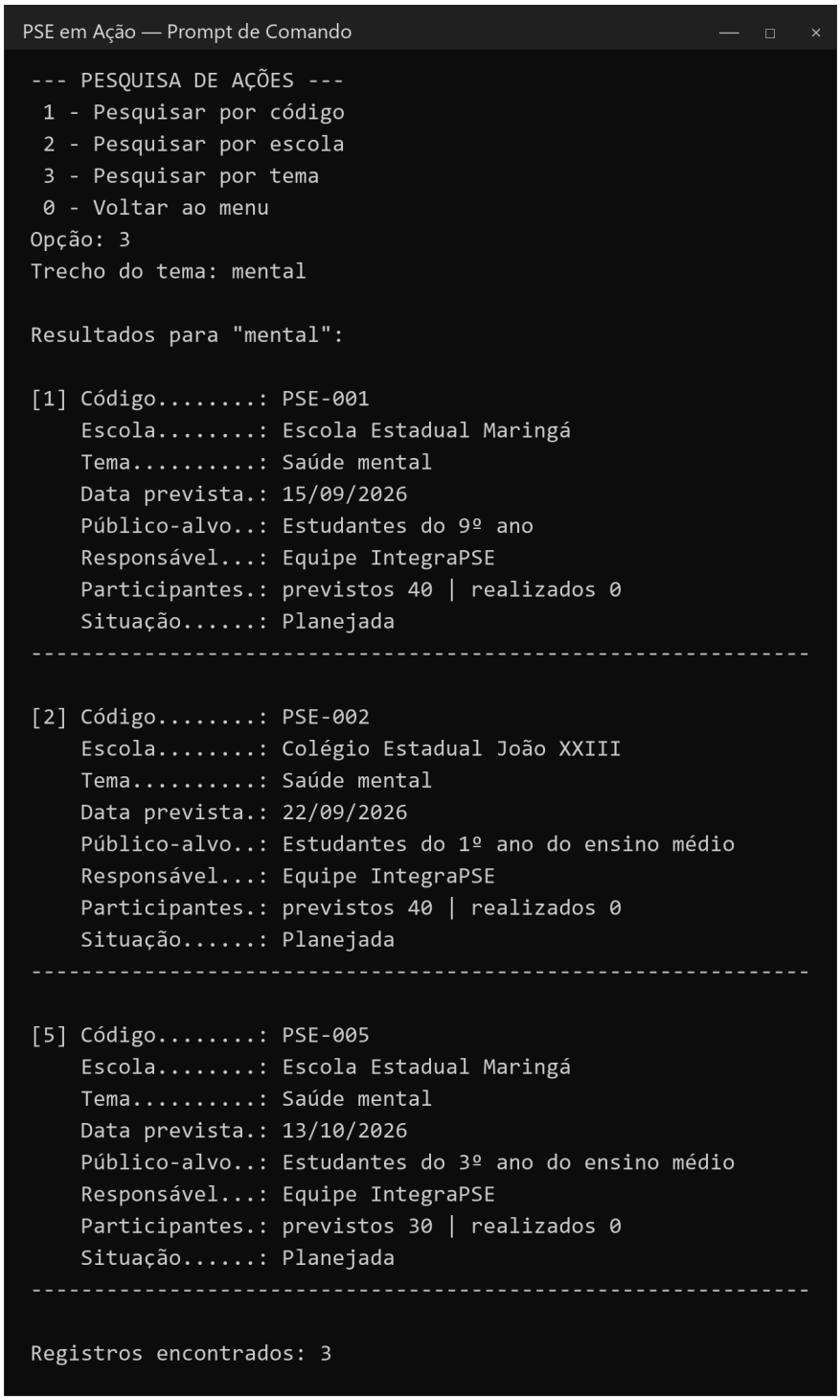


Figura 8 – Pesquisa de ações por tema. Fonte: elaboração própria (2026).

7.6.6 Atualização de situação

A opção 4 localiza a ação pelo código, exibe os dados atuais e solicita a nova situação. Quando a ação é marcada como realizada, o sistema solicita a quantidade efetiva de participantes, conforme a figura 9.

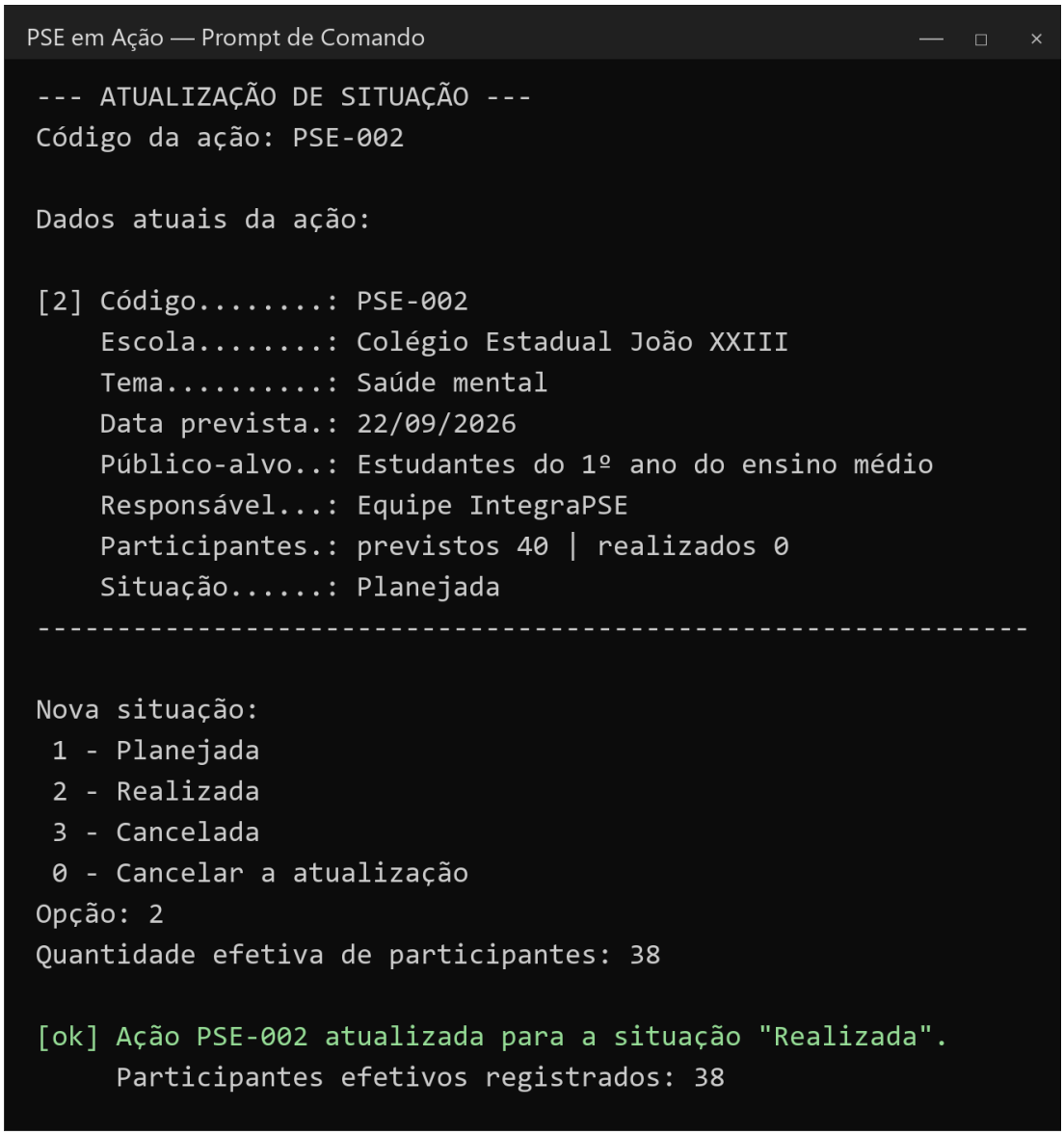


Figura 9 – Atualização de situação com registro de participantes. Fonte: elaboração própria (2026).

7.6.7 Registro inexistente

Quando o código informado não corresponde a nenhum registro, o sistema informa a inexistência da ação e orienta o usuário a conferir o código na listagem, como mostra a figura 10.

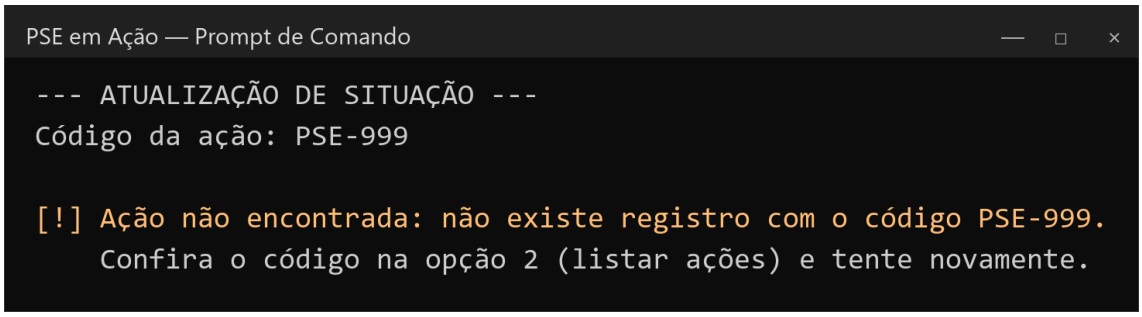


Figura 10 – Mensagem para registro inexistente. Fonte: elaboração própria (2026).

7.6.8 Resumo geral

A opção 5 consolida os dados das ações cadastradas. A figura 11 apresenta o resumo de cinco ações, com duas planejadas, duas realizadas e uma cancelada, o total de participantes previstos e efetivos e o percentual de participação.

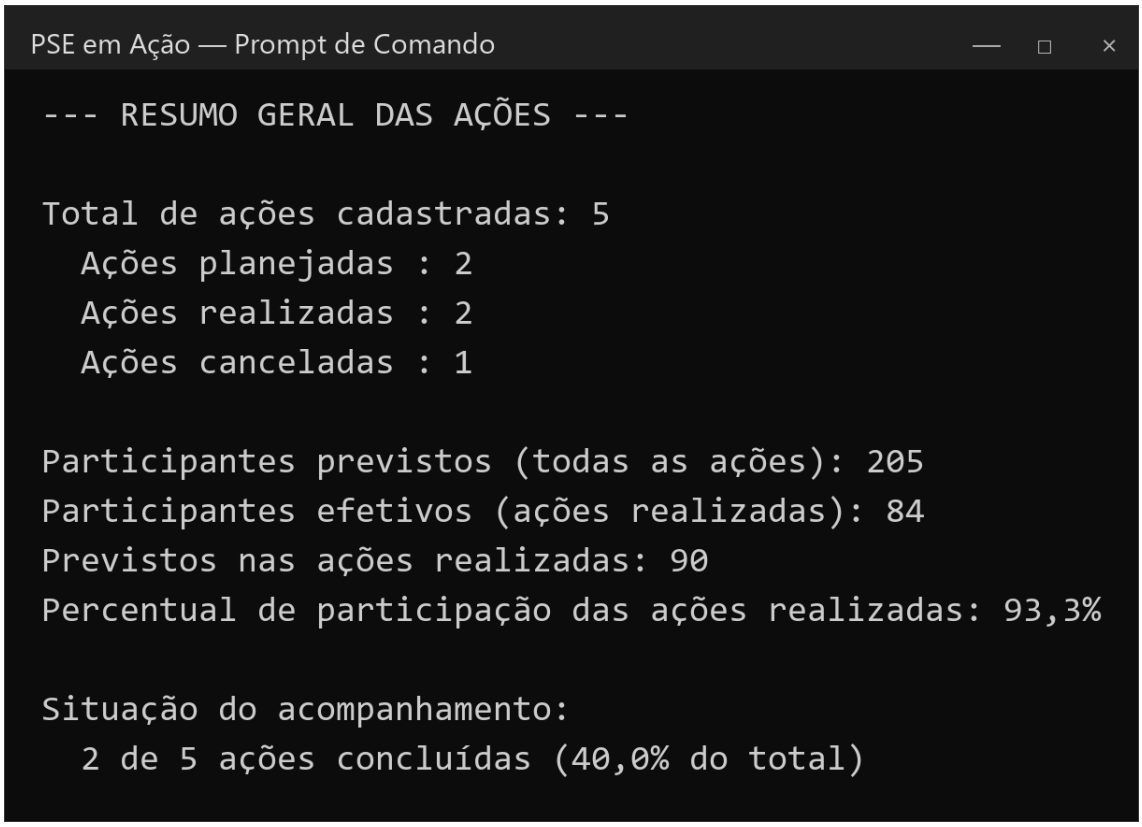


Figura 11 – Resumo geral das ações. Fonte: elaboração própria (2026).

7.7 Encerramento

Para encerrar o sistema, o usuário digita 0 no menu principal. O programa exibe a mensagem de encerramento e finaliza. Como os dados são mantidos apenas em memória, as informações cadastradas são perdidas ao encerrar a

execução, comportamento previsto no escopo do projeto e discutido nas considerações finais.

8 CONSIDERAÇÕES FINAIS

8.1 Resultados alcançados

O sistema PSE em Ação foi implementado em linguagem C e atende às seis funcionalidades mínimas do desafio: cadastro de ações, listagem, pesquisa por código, escola ou tema, atualização de situação com registro de participantes, geração de resumo geral e validação das entradas. O código compila sem erros nem advertências com as opções -Wall -Wextra e executa em terminal, sem dependências externas.

O resultado atende ao objetivo geral definido na primeira entrega, que era desenvolver um sistema capaz de planejar, registrar e acompanhar ações coletivas do PSE, contribuindo para a organização das informações entre a escola e a equipe de saúde. A consolidação automática dos totais e do percentual de participação substitui o trabalho manual de reunir planilhas separadas, que era a origem do problema identificado.

8.2 Limitações

A principal limitação do sistema é o armazenamento apenas em memória: ao encerrar o programa, os registros são perdidos. Essa característica foi prevista no escopo do trabalho e é adequada ao nível introdutório, mas limita o uso real por uma equipe que precise manter histórico entre sessões.

Também foram identificadas como limitações: o limite fixo de 100 registros; a ausência de funções de exclusão e de edição dos demais campos de uma ação, permitindo apenas a alteração da situação; o armazenamento da data como texto, sem ordenação cronológica; e a inexistência de relatórios por escola ou por tema, que seriam úteis para o acompanhamento do programa.

8.3 Aprendizados

O desenvolvimento permitiu aplicar, em um problema concreto, os conteúdos trabalhados no semestre: estruturas condicionais, laços de repetição, vetores, estruturas (struct) e funções. A separação das responsabilidades em funções foi o aprendizado mais significativo, pois mostrou que um programa pode crescer sem se tornar difícil de manter quando cada função tem um papel claro.

Outro aprendizado relevante foi a validação das entradas. Nos primeiros testes, entradas inválidas faziam o programa se comportar de forma inesperada, o que evidenciou a importância de tratar os dados antes de armazená-los. O grupo também percebeu que a organização do trabalho em etapas semanais, definida na primeira entrega, foi decisiva para concluir a implementação e a documentação dentro do prazo.

8.4 Possibilidades de melhoria

Como melhorias futuras, destacam-se: a persistência dos dados em arquivo, prevista como melhoria opcional no desafio; a inclusão de funções de edição e exclusão de registros; a ordenação das ações por data prevista; a geração de relatórios por escola e por tema; a exportação do resumo em formato de texto ou CSV; e a substituição do armazenamento em vetor por estruturas mais adequadas a grandes volumes de dados.

Do ponto de vista de uso, uma evolução natural seria permitir o registro do planejamento anual de cada escola, com acompanhamento por período, aproximando o sistema da rotina real da equipe intersetorial do Programa Saúde na Escola.

9 LINKS DE ACESSO

O código-fonte completo, o README com as instruções de compilação e execução e a documentação das duas etapas estão disponíveis no repositório do projeto no GitHub:

https://github.com/volpszz/aep_2semestre

A apresentação em vídeo, com duração de até sete minutos, demonstra o sistema em execução, explica as decisões de implementação e relaciona os requisitos, os fluxogramas e as funcionalidades implementadas:

[LINK DO VÍDEO – inserir o link após a gravação, com acesso liberado para avaliação]

REFERÊNCIAS

BRASIL. Decreto nº 6.286, de 5 de dezembro de 2007. Institui o Programa Saúde na Escola – PSE e dá outras providências. Diário Oficial da União: Brasília, DF, 6 dez. 2007.

BRASIL. Ministério da Saúde. O que é o Programa Saúde na Escola (PSE)? Brasília, DF, 2025a. Disponível no portal do Ministério da Saúde. Acesso em: 24 jul. 2026.

BRASIL. Ministério da Saúde. Quais são as ações do PSE? Brasília, DF, 2025b. Disponível no portal do Ministério da Saúde. Acesso em: 24 jul. 2026.

G1. Crise de saúde mental: Brasil tem maior número de afastamentos por ansiedade e depressão em 10 anos. 2025. Disponível no portal G1. Acesso em: 5 set. 2026.

INSTITUTO AYRTON SENNA. Pesquisa mostra que 79% dos alunos no final do ensino médio sofrem algum sintoma de depressão ou ansiedade. Avaliação do Futuro, em parceria com o Centro de Políticas Públicas e Avaliação da Educação (CAEd/UFJF). 2025. Disponível no portal do Instituto Ayrton Senna. Acesso em: 5 set. 2026.

SOUSA, Marta Caires de; ESPERIDIÃO, Monique Azevedo; MEDINA, Maria Guadalupe. A intersetorialidade no Programa Saúde na Escola: avaliação do processo político-gerencial e das práticas de trabalho. Ciência & Saúde Coletiva, Rio de Janeiro, v. 22, n. 6, p. 1781-1790, 2017. DOI: 10.1590/1413-81232017226.24262016.

APÊNDICE A – FLUXOGRAMAS DO SISTEMA

Os fluxogramas a seguir foram elaborados na primeira entrega e serviram de base para a implementação do sistema. Eles são reproduzidos nesta etapa como parte dos artefatos revisados, permitindo comparar o que foi planejado com o que foi efetivamente implementado.

A figura A1 apresenta o fluxo geral: o menu principal é exibido repetidamente, a opção informada é validada e cada escolha chama a função correspondente, até que o usuário digite 0 para encerrar. A estrutura implementada corresponde ao fluxograma — a função main controla o laço de repetição e delega cada operação a uma função específica.

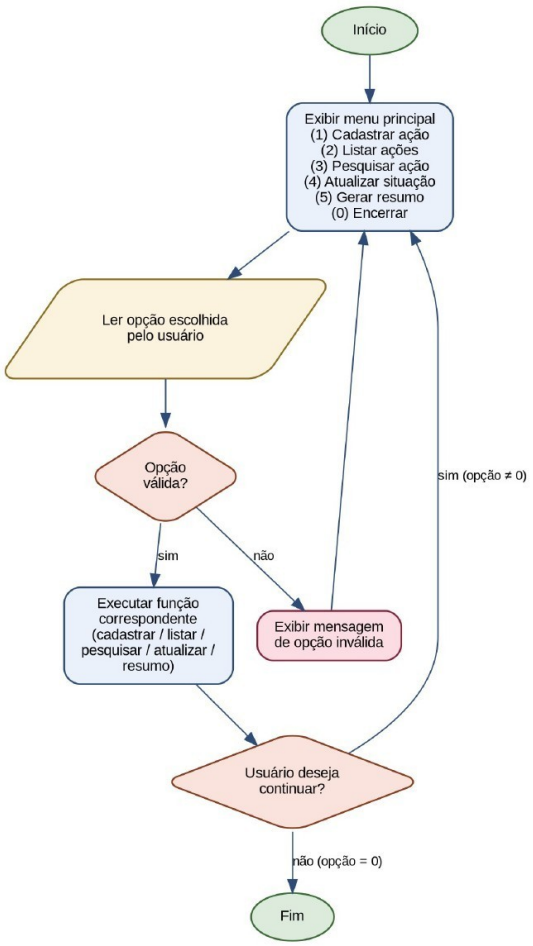


Figura A1 – Fluxograma geral do sistema PSE em Ação. Fonte: elaboração própria (2026).

A figura A2 detalha o cadastro de uma ação, com as decisões de código duplicado, campo obrigatório vazio e quantidade negativa. Em relação ao planejado, a implementação acrescentou a validação da data no calendário e passou a solicitar

a situação inicial ao usuário, em vez de defini-la automaticamente como “planejada”. As duas alterações estão registradas na tabela 3 deste documento.

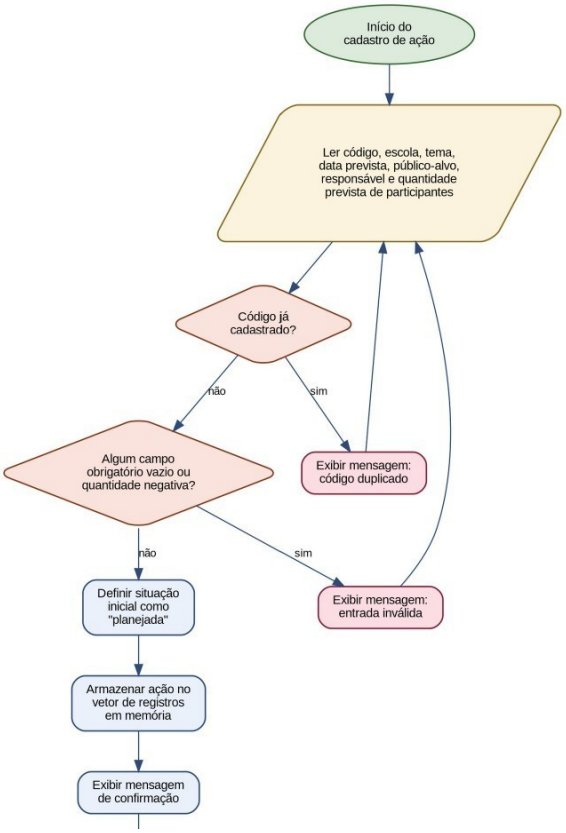


Figura A2 – Fluxograma detalhado do cadastro de uma ação. Fonte: elaboração própria (2026).